

uart_cmd

[toc]

项目简介

uart_cmd文件夹下的sample提供了在windows和linux平台下使用uart通路接发命令的示例。

该示例展示了以下功能：

- 固件更新
- 发送命令

该示例从上层（应用层）到下层（通信协议层）分别依赖了：

基本功能组件（commom文件夹）中的data模块

特殊功能组件（components文件夹）中的cmd模块

接口适配层（interfaces文件夹）中的libircam\libircmd模块

驱动层（drivers文件夹）中的libiruart模块

设备连接

机芯通过UART0串口发送命令，通过UART1串口读取运行码（运行时的log信息）。

若需使用uart示例接发命令，需要连接UART0线。

程序编译

windows平台

有两种编译方式

使用Visual Studio进行编译

- 将程序编译时所需的libircmd.lib\libircam.lib\libiruart.lib\opencv_world455.lib\pthreadVC2.lib五个静态库放入libs\x64\dll文件夹下
- 进入msvc文件夹下，双击打开.sln文件
- 勾选合适的编译环境（目前仅支持release|x64和release|x86两种编译环境）

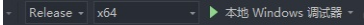


image-20230227145452287

- 打开解决方案资源管理器（在顶部菜单栏视图选项中），按照下图操作，在Release/x64文件夹下可生成可执行文件



image-20230308111929169

- 将程序运行时所需的libircmd.dll\libircam.dll\libiruart.dll三个动态库放入Release/x64文件夹下，双击sample.exe即可运行该可执行程序

RS300_SDK_libir_sample > sample > uart_cmd > Release > x64				
搜索"x64"				
名称	修改日期	类型	大小	
libircam.dll	2023/1/11 15:32	应用程序扩展	12 KB	
libircmd.dll	2023/2/6 11:14	应用程序扩展	59 KB	
libiruart.dll	2023/3/3 13:49	应用程序扩展	25 KB	
sample.exe	2023/3/8 11:19	应用程序	24 KB	
sample.pdb	2023/3/8 11:19	Program Debug...	548 KB	

image-20230308112048790

使用bat脚本进行编译

uart_cmd文件夹下提供了compile.bat批处理脚本，支持使用VisualStudio编译Release|x64以及Release|x86环境下的sample程序，并将可执行程序运行所需的动态库同时拷贝到可执行文件的生成路径下。

- 找到devenv.exe文件的存储路径，并配置系统环境变量

<< Microsoft Visual Studio > 2019 > Community > Common7 > IDE				
搜索"IDE"				
名称	修改日期	类型	大小	
api-ms-win-crt-stdio-l1-1-0.dll	2022/1/19 8:58	应用程序扩展	24 KB	
api-ms-win-crt-string-l1-1-0.dll	2022/1/19 8:58	应用程序扩展	24 KB	
api-ms-win-crt-time-l1-1-0.dll	2022/1/19 8:58	应用程序扩展	21 KB	
api-ms-win-crt-utility-l1-1-0.dll	2022/1/19 8:58	应用程序扩展	19 KB	
Automation.manifest	2022/1/19 8:55	MANIFEST 文件	23 KB	
CodeMarkersEtw.man	2022/1/19 8:58	MAN 文件	1 KB	
CodeMarkersEtwRc.dll	2022/1/19 8:58	应用程序扩展	743 KB	
complusm.dll	2022/1/19 9:02	应用程序扩展	396 KB	
concr140.dll	2022/1/19 8:58	应用程序扩展	240 KB	
DbgComposition.dll	2022/1/19 9:02	应用程序扩展	345 KB	
DDConfigCA.exe	2022/1/19 8:57	应用程序	144 KB	
devenv.com	2022/1/19 8:58	MS-DOS 应用程序	8 KB	
devenv.exe	2022/9/5 18:53	应用程序	733 KB	
devenv.exe.config	2022/1/19 8:58	Configuration 源...	94 KB	
devenv.isolation.ini	2022/9/5 18:53	配置设置	2 KB	
devenv.prf	2022/1/19 8:58	PICS Rules File	26 KB	
devenv.visualelementsmanifest.xml	2022/1/19 8:58	XML 文档	1 KB	

image-20230228110100441

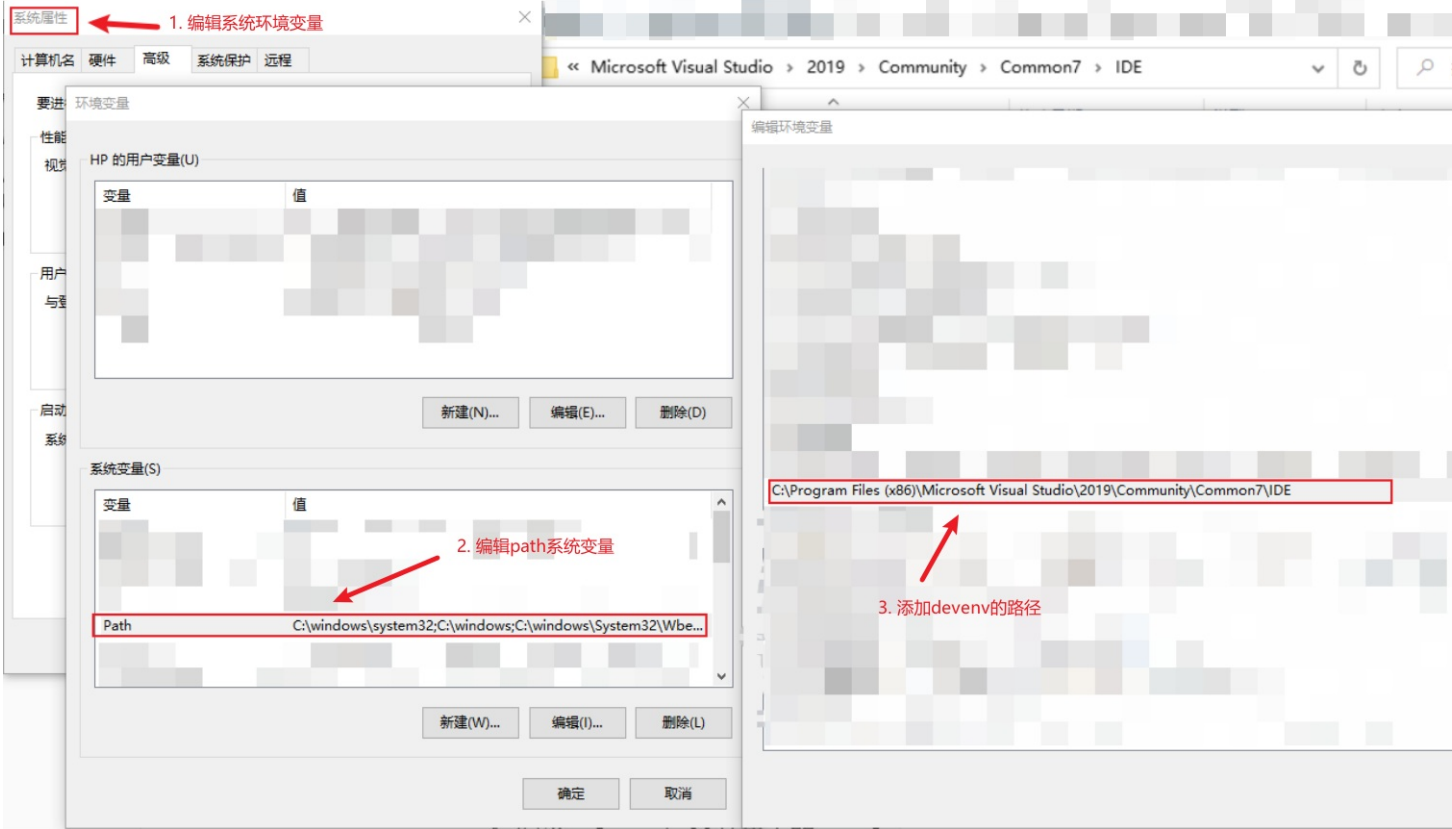


image-20230228112910504

2. 将libircmd.lib\libircam.lib\libiruart.lib\libircmd.dll\libircam.dll\libiruart.dll六个库放入libs\x64\dll文件夹下
3. 在uart_cmd文件夹下，使用命令提示符，根据需求输入如下的编译命令
 1. 编译Release|x86环境下的sample
compile.bat x86
 2. 编译Release|x64环境下的sample
compile.bat x64
4. 当出现以下提示时，说明编译成功，可执行程序及其运行时所需的动态库在Release\x64文件夹下

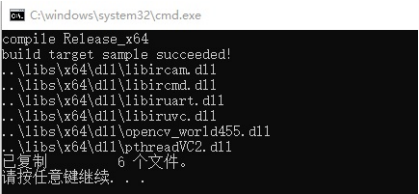


image-20230308112552667

5. 进入Release\x64文件夹，双击sample.exe即可运行该可执行程序

名称	修改日期	类型	大小
libircam.dll	2023/1/11 15:32	应用程序扩展	12 KB
libircmd.dll	2023/2/6 11:14	应用程序扩展	59 KB
libiruart.dll	2023/3/3 13:49	应用程序扩展	25 KB
sample.exe	2023/3/8 11:19	应用程序	24 KB
sample.pdb	2023/3/8 11:19	Program Debug...	548 KB

image-20230308112048790

Linux平台

uart_cmd文件夹下提供了CMakeList.txt文件，支持32位或64位sample程序的编译，相应的编译命令如下：

编译32位的sample

```
cmake -DBUILD32=ON
make
```

编译64位的sample

```
cmake -DBUILD32=OFF
make
```

注意：在sample编译之前，需要事先获取与sample相同位数（32位或64位）的libircam.so\libircmd.so\libiruart.so三个动态库，并将其放于drivers文件夹下，否则直接使用CMakeList.txt编译sample时会提示错误。

功能展示

串口连接

运行SDK后，SDK支持使用两种方式连接可用的串口。

- 事先知道UART0设备的串口编号，在sample.cpp中对com_number字段的值进行修改，eg：需要打开COM17，则应将红框字段改为com_number = 17（使用环境：需要烧录固件，知道UART0串口编号）

```
147     int ret;
148     int com_number = 0;
149     ret = search_com(ir_control_handle, com_number);
150     if (ret != 0)
151     {
152         printf("fail to find available com\n");
153         return -1;
154     }
```

image-20230308152700668

- 并不知道UART0设备的串口编号，程序会自动寻找并连接可用的UART0（使用环境：已烧录固件，但是不清楚UART0串口编号）

```
uart cmd sample start
version:develop
version:libircmd 0.4.1.alpha2
version:libircam 0.2.0.alpha
libircmd debug [D:\WORK\RS300\first_packet\gerriit\RS300_SDK_libircmd\src\libircmd.c:109\ircmd_create_handle]
正在搜寻当前可用的串口.....
success to open COM19
-----please select cmd type-----
1:get_device_info
2:basic_ffc_update
3:palette_operate
4:temp_measure
5:data_update
6:set_image_scene_mode
7:image_effection
8:firmware upgrade
-----
```

image-20230308151707867

- 串口波特率设置，在search_com函数内，调用ir_control_open插件后，调用ir_control_init插件设置波特率，
 - 对于未调用ir_control_init插件设置波特率的情况，SDK内部将默认使用115200波特率与机芯交互
 - 调用ir_conteol_init插件第二个参数可以设置为NULL，此时将会自动适配波特率，如果自动适配失败将会返回错误码IRUART_SET_BAUDRATE_FAILED:-523
 - 建议在清楚固件波特率的情况下，使用正确波特率作为参数调用ir_conteol_init插件，如

```
int device_params[64] = { 0 };
device_params[0] = 921600;
ir_control_handle->ir_control_init(ir_control_handle->ir_control_handle, device_params);
```

 - 如果固件是921600波特率，在手动调用了“basic_reset_to_rom”接口进入rom状态后，需要调用ir_conteol_init插件设置波特率为115200，退出后重新设置波特率为921600

固件更新

该示例提供更新内部flash或外部flash中的固件的功能

固件更新的流程如下：

- 键入8触发固件更新功能

```
uart cmd sample start
version:develop
version:libircmd 0.4.1.alpha2
version:libircam 0.2.0.alpha
正在搜寻当前可用的串口.....
COM3 exist, but not uart0
success to open COM19
-----please select cmd type-----
1:get_device_info
2:basic_ffc_update
3:palette_operate
4:temp_measure
5:data_update
6:set_image_scene_mode
7:image_effection
8:firmware upgrade
-----
8
please enter local firmware file path:
../dataset/0.7.0.bin
```

image-20230308155637388

- 看到please enter local firmware file path:提示后输入需要写入的固件程序的绝对路径，或相对于可执行文件sample.exe的相对路径

rom程序	文件名	固件将被烧录到的区域
内部rom	文件名以EXT.bin结尾	外部flash
内部rom	文件名以RAM.bin结尾	SRAM
内部rom	文件名非EXT.bin或RAM.bin结尾	内部flash
外部rom	任意文件名	外部flash

- 进入固件数据写入阶段，若定义回调函数，程序会实时显示当前固件数据写入的进度

```
60     IrcmdError_e ret;
61     ret = basic_firmware_download(cmd_handle, basename, firmware_file_data, local_file_length, process_cb);
62     if (ret != IRCMD_SUCCESS)
63     {
64         printf("firmware_download failed\n");
65         free(firmware_file_data);
66         return;
67     }
```

image-20230308155953833

- 完成固件更新后，固件程序会自动重新加载启动

发送命令

键入1~7命令，触发相应的命令接收，详情可见cmd.cpp

```
//command thread function
void* cmd_function(void* threadarg)
{
    int isRUNNING = 1;
    int cmd;
    while (isRUNNING)
    {
        printf("please select cmd type\n");
        printf("1:get_device_info 2:basic_ffc_update 3:palette_operate 4:temp_measure 5:data_update\n");
        printf("6:set_image_scene_mode 7:image_effection\n");
        scanf("%d", &cmd);
        command_sel(cmd, (StreamFrameInfo_t*)threadarg);
    }
    printf("cmd thread exit!!\n");
    return NULL;
}
```